

Student Academic Manager

A Flutter application for managing student courses and homework assignments with real-time Firebase Firestore synchronization.

Table of Contents

- [Project Overview](#)
 - [Features](#)
 - [Project Structure](#)
 - [Architecture](#)
 - [Setup Instructions](#)
 - [Code Documentation](#)
 - [API Reference](#)
 - [Firebase Configuration](#)
 - [Usage Examples](#)
 - [Contributing](#)
-

Project Overview

Student Academic Manager is a mobile and web application built with Flutter that enables students to organize and manage their academic workload. It provides a centralized platform to track courses, manage homework assignments, and monitor deadlines.

Target Users: Students (primary), Teachers (secondary)

Core Functionality:

- Create, read, update, and delete courses
 - Manage homework assignments linked to courses
 - Track homework completion status
 - Real-time synchronization with Firebase Firestore
 - Material Design 3 UI with modern animations
-

Features

Current Features 

- **Course Management**
 - Add/Edit/Delete courses
 - Store course details (name, teacher, day, time)
 - Real-time course list updates
- **Homework Management** (Backend ready)
 - Create homework assignments

- Link homework to courses
- Mark homework as complete/incomplete
- Filter pending and overdue assignments

• Real-Time Sync

- StreamBuilder for live Firestore updates
- Automatic data synchronization across devices

• Error Handling

- Custom exception system with error codes
- User-friendly error messages
- Loading and empty states

Planned Features 🚀

- Homework UI screens (list and form)
- Authentication (email/password, Google Sign-In)
- Notifications for due dates
- Calendar view integration
- Export reports to PDF
- Dark mode support
- Multi-language support

📁 Project Structure

```
task_2/
├── lib/
│   ├── main.dart                # App entry point & Firebase init
│   ├── models/
│   │   ├── course.dart          # Course data model
│   │   ├── homework.dart        # Homework data model
│   │   └── index.dart            # Model exports
│   ├── services/
│   │   ├── firestore_service.dart # Base Firestore operations (legacy)
│   │   ├── course_service.dart   # Course CRUD & business logic
│   │   ├── homework_service.dart # Homework CRUD & business logic
│   │   └── index.dart            # Service exports
│   ├── screens/
│   │   ├── course_list_screen.dart # Display all courses
│   │   ├── course_form_screen.dart # Add/Edit course form
│   │   └── index.dart            # Screen exports
│   └── exceptions/
│       ├── app_exception.dart    # Custom exception class
│       └── index.dart            # Exception exports
├── android/                      # Android native code
│   ├── app/
│   │   ├── google-services.json  # Firebase Android config (DO NOT COMMIT)
│   │   └── build.gradle.kts
```

```

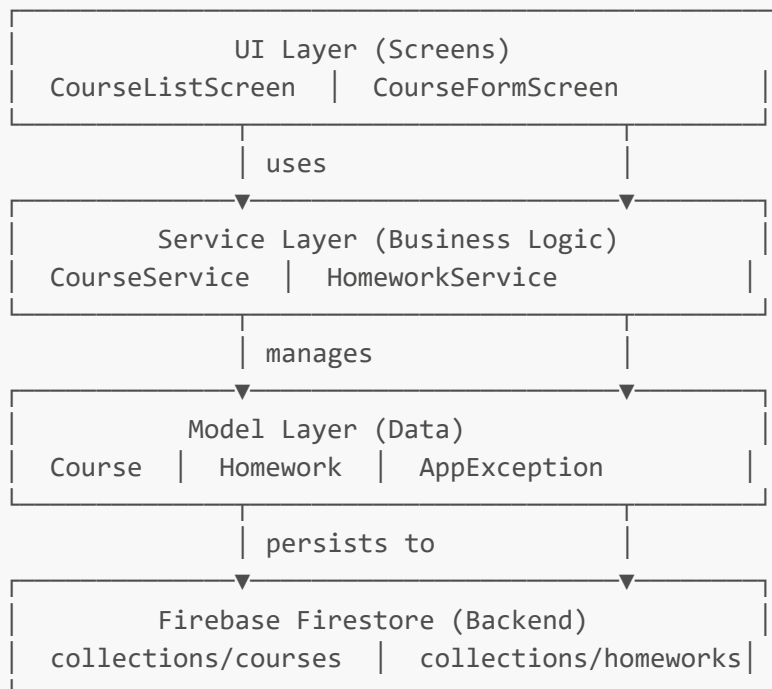
├── build.gradle.kts
├── ios/                # iOS native code
├── pubspec.yaml        # Flutter dependencies
├── analysis_options.yaml # Lint rules
└── README.md          # This file

```



Architecture

Architecture Pattern: **MVSC (Model-View-Service-Controller)**



Design Patterns Used

1. Singleton Pattern (Services)

- `CourseService` and `HomeworkService` use singleton to maintain single instance
- Prevents multiple Firestore connections

2. Factory Constructor (Models)

- `Course.fromMap()` and `Homework.fromMap()` convert Firestore data to Dart objects
- Ensures type safety and null safety

3. StreamBuilder (UI)

- Real-time data binding from Firestore
- Automatic UI updates when data changes

4. Custom Exceptions

- `AppException` with error codes for proper error handling
- Wraps all async operations in try-catch

Setup Instructions

Prerequisites

- Flutter SDK 3.11.4+
- Dart 3.11.4+
- Firebase project
- Android/iOS device or emulator
- Git

1. Clone Repository

```
git clone <repository-url>
cd task_2
```

2. Install Dependencies

```
flutter pub get
```

3. Configure Firebase

Option A: Auto Configuration (Recommended)

```
# Install FlutterFire CLI
flutter pub global activate flutterfire_cli

# Configure for your Firebase project
flutterfire configure --overwrite
```

Option B: Manual Configuration

Follow [Firebase Configuration](#) section below.

4. Run the App

On Android Phone/Emulator:

```
flutter run
```

On Web:

```
flutter run -d chrome
```

Release Build:

```
flutter run --release
```

5. Verify Firebase Connection

- Open the app
- Try adding a course
- Check Firebase Console → Firestore Database → **courses** collection
- You should see the new document



 Code Documentation

Models

Course Model (**lib/models/course.dart**)

```
class Course {
  final String id;           // Firestore document ID
  final String name;         // Course name (e.g., "Mathematics")
  final String teacher;      // Teacher name
  final String day;          // Day of week (Monday-Saturday)
  final String time;         // Time in HH:mm format
  final DateTime? createdAt; // Auto-set on creation
  final DateTime? updatedAt; // Auto-updated on modification
}
```

Key Methods:

Method	Purpose	Example
fromMap()	Convert Firestore doc to Course	<code>Course.fromMap(data, docId)</code>
toMap()	Convert Course to Firestore format	<code>firestore.add(course.toMap())</code>
copyWith()	Create modified copy	<code>course.copyWith(name: 'Physics')</code>

Null Safety:

- All required fields are non-nullable
- **createdAt** and **updatedAt** are nullable (optional metadata)

Homework Model (`lib/models/homework.dart`)

```
class Homework {  
  final String id;           // Firestore document ID  
  final String title;        // Homework title  
  final DateTime deadline;    // Due date  
  final bool isDone;          // Completion status  
  final String courseId;      // Foreign key to Course  
  final String? description;  // Optional details  
  final DateTime? createdAt;  // Auto-set on creation  
  final DateTime? updatedAt;  // Auto-updated on modification  
}
```

Important:

- `courseId` links homework to a course
- `deadline` is required (no null value)
- Default deadline is 7 days if not specified

Services

CourseService (`lib/services/course_service.dart`)

Handles all course operations with Firestore.

Singleton Access:

```
final courseService = CourseService();  
// Returns the same instance every call
```

Core CRUD Methods:

```
// 1. Get all courses (real-time stream)  
courseService.getAll().listen((courses) {  
  print('Courses updated: ${courses.length}');  
});  
  
// 2. Get single course  
final course = await courseService.getById('course123');  
  
// 3. Add new course  
final courseId = await courseService.add(  
  Course(  
    id: '',  
    name: 'Mathematics',  
    teacher: 'Mr. Smith',  
  ),  
);
```

```
        day: 'Monday',
        time: '08:00',
      ),
    );

// 4. Update course
await courseService.update(updatedCourse);

// 5. Delete course
await courseService.delete('course123');
```

Additional Methods:

```
// Search courses by teacher name
courseService.searchByTeacher('Smith')
  .listen((filteredCourses) { });

// Get courses for specific day
courseService.getCoursesByDay('Monday')
  .listen((mondayCourses) { });
```

Error Handling:

```
try {
  await courseService.add(course);
} on AppException catch (e) {
  print('Error: ${e.message} (Code: ${e.code})');
}
```

HomeworkService ([lib/services/homework_service.dart](#))

Handles all homework operations with Firestore.

Core CRUD Methods:

```
// Get all homeworks
homeworkService.getAll().listen((homeworks) { });

// Get by ID
final hw = await homeworkService.getById('hw123');

// Add homework
final hwId = await homeworkService.add(homework);

// Update homework
await homeworkService.update(updatedHomework);
```

```
// Delete homework
await homeworkService.delete('hw123');
```

Special Methods:

```
// 🔥 Get homeworks for specific course
homeworkService.getHomeworksByCourse('course123')
  .listen((courseHomeworks) { });

// Get pending (not done + future deadline)
homeworkService.getPendingHomeworks()
  .listen((pending) { });

// Get overdue (not done + past deadline)
homeworkService.getOverdueHomeworks()
  .listen((overdue) { });

// Get completed
homeworkService.getCompletedHomeworks()
  .listen((done) { });

// Quick status toggle
await homeworkService.toggleStatus('hw123', true);
```

Screens

CourseListScreen (lib/screens/course_list_screen.dart)

Displays all courses with real-time updates.

Key Features:

1. **StreamBuilder** - Listens to `CourseService.getAll()`

```
StreamBuilder<List<Course>>(
  stream: _courseService.getAll(),
  builder: (context, snapshot) {
    // Rebuilds UI when data changes
  }
)
```

2. **Course Card Item**

- Shows: name, teacher, day, time
- Edit button → Navigate to CourseFormScreen
- Delete button → Confirmation dialog

3. States

- **Loading**: CircularProgressIndicator
- **Error**: Error message with icon
- **Empty**: Helpful message with FAB hint
- **Loaded**: ListView of course cards

4. Floating Action Button - Navigate to add new course

UI Flow:

```
CourseListScreen
├─ [Loading State] → CircularProgressIndicator
├─ [Error State] → Error message + retry
├─ [Empty State] → "No Courses Yet" message
└─ [Data State] → ListView
    ├── CourseCard 1
    │   ├── Title: Mathematics
    │   ├── Subtitle: Teacher, Day • Time
    │   ├── Edit Button
    │   └── Delete Button
    ├── CourseCard 2
    └── CourseCard N
```

CourseFormScreen ([lib/screens/course_form_screen.dart](#))

Add or edit courses with form validation.

Dual Mode:

```
// Add mode (new course)
CourseFormScreen()

// Edit mode (existing course)
CourseFormScreen(course: existingCourse)
```

Form Fields:

Field	Type	Validation
Name	TextFormField	Required, min 2 chars
Teacher	TextFormField	Required, min 2 chars
Day	DropDownButtonFormField	Required (Mon-Sat)
Time	TextFormField	Required, HH:mm format

Validation Example:

```
// Time format validation using regex
bool _isValidTimeFormat(String time) {
  final regex = RegExp(r'^([01]?[0-9]|2[0-3]):[0-5][0-9]$');
  return regex.hasMatch(time);
}
// Valid: "08:00", "14:30", "23:59"
// Invalid: "25:00", "8:0", "8:00am"
```

Save Flow:

```
User clicks Save
↓
Validate form
  ↓ (if valid)
Show loading spinner
  ↓
Call service.add() or service.update()
  ↓ (if success)
Show success Snackbar
  ↓
Pop screen (return to list)
  ↓ (if error)
Show error Snackbar
  ↓
Keep form open for retry
```

Error Handling:

```
try {
  final course = Course(...);
  if (isEditMode) {
    await _courseService.update(course);
  } else {
    await _courseService.add(course);
  }
  // Success
  Navigator.pop(context);
} on AppException catch (e) {
  // Show error message
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Error: ${e.message}'))
  );
}
```

Exception Handling

AppException ([lib/exceptions/app_exception.dart](#))

Custom exception for uniform error handling.

```
class AppException implements Exception {
  final String message;          // User-friendly message
  final String? code;            // Error code for logging
  final dynamic originalError;   // Original exception for debugging
}
```

Usage:

```
// Throw exception
throw AppException(
  message: 'Failed to add course',
  code: 'ADD_COURSE_ERROR',
  originalError: firebaseError,
);

// Catch and handle
try {
  await courseService.add(course);
} on AppException catch (e) {
  if (e.code == 'ADD_COURSE_ERROR') {
    // Handle specific error
  }
  print(e); // Prints: "AppException: Failed to add course (ADD_COURSE_ERROR)"
}
```

Error Codes Reference:

Code	Meaning
FETCH_COURSES_ERROR	Failed to fetch all courses
FETCH_COURSE_BY_ID_ERROR	Failed to fetch single course
ADD_COURSE_ERROR	Failed to add new course
UPDATE_COURSE_ERROR	Failed to update course
DELETE_COURSE_ERROR	Failed to delete course
INVALID_COURSE_ID	Empty or invalid course ID
SEARCH_BY_TEACHER_ERROR	Search query failed



API Reference

CourseService

```
// Real-time stream of all courses
Stream<List<Course>> getAll()

// Fetch single course (one-time, not streaming)
Future<Course?> getById(String id)

// Create new course
Future<String> add(Course course)

// Update existing course
Future<void> update(Course course)

// Delete course by ID
Future<void> delete(String id)

// Search by teacher name (client-side filtering)
Stream<List<Course>> searchByTeacher(String teacherName)

// Filter by day of week
Stream<List<Course>> getCoursesByDay(String day)
```

HomeworkService

```
// Real-time stream of all homeworks
Stream<List<Homework>> getAll()

// Fetch single homework
Future<Homework?> getById(String id)

// Create new homework
Future<String> add(Homework homework)

// Update existing homework
Future<void> update(Homework homework)

// Delete homework by ID
Future<void> delete(String id)

// ★ Get homeworks for specific course
Stream<List<Homework>> getHomeworksByCourse(String courseId)

// Get not-done with future deadline
Stream<List<Homework>> getPendingHomeworks()

// Get not-done with past deadline
Stream<List<Homework>> getOverdueHomeworks()

// Get completed homeworks
Stream<List<Homework>> getCompletedHomeworks()
```

```
// Toggle completion status
Future<void> toggleStatus(String id, bool isDone)

// Get pending for specific course
Stream<List<Homework>> getPendingHomeworksByCourse(String courseId)
```

Firebase Configuration

Firestore Database Structure

```
Firestore
├── courses (collection)
│   └── {documentId}
│       ├── name: "Mathematics" (string)
│       ├── teacher: "Mr. Smith" (string)
│       ├── day: "Monday" (string)
│       ├── time: "08:00" (string)
│       ├── createdAt: "2024-05-03T10:00:00.000Z" (timestamp)
│       └── updatedAt: "2024-05-03T10:00:00.000Z" (timestamp)
└── homeworks (collection)
    └── {documentId}
        ├── title: "Chapter 5 Exercises" (string)
        ├── deadline: "2024-05-10T18:00:00.000Z" (timestamp)
        ├── isDone: false (boolean)
        ├── courseId: "{courseDocId}" (string - foreign key)
        ├── description: "Pages 45-50" (string, optional)
        ├── createdAt: "2024-05-03T10:00:00.000Z" (timestamp)
        └── updatedAt: "2024-05-03T10:00:00.000Z" (timestamp)
```

Firestore Rules (Security)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Allow all reads and writes (development only)
    // ⚠️ CHANGE THIS IN PRODUCTION!
    match /{document=**} {
      allow read, write: if true;
    }

    // Production rule example:
    // match /courses/{document=**} {
    //   allow read: if request.auth != null;
    //   allow write: if request.auth != null && request.auth.uid ==
    resource.data.userId;
    // }
  }
}
```

```
}  
}
```

Android Configuration File

Place `google-services.json` in `android/app/`:

```
{  
  "project_info": {  
    "project_number": "123456789",  
    "project_id": "student-companion-88ade",  
    "storage_bucket": "student-companion-88ade.appspot.com"  
  },  
  "client": [  
    {  
      "client_info": {  
        "mobilesdk_app_id": "1:123456789:android:abc123def456",  
        "android_client_info": {  
          "package_name": "com.example.task_2"  
        }  
      },  
      "api_key": [  
        {  
          "current_key": "AIzaSyD..."  
        }  
      ]  
    }  
  ]  
}
```

iOS Configuration File

Place `GoogleService-Info.plist` in `ios/Runner/`:

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE plist PUBLIC...>  
<plist version="1.0">  
  <dict>  
    <key>PROJECT_ID</key>  
    <string>student-companion-88ade</string>  
    <key>BUNDLE_ID</key>  
    <string>com.example.task2</string>  
    ...  
  </dict>  
</plist>
```

Add a New Course

```
// 1. Create course object
final newCourse = Course(
  id: '', // Will be auto-generated by Firestore
  name: 'Physics',
  teacher: 'Dr. Johnson',
  day: 'Wednesday',
  time: '14:30',
);

// 2. Call service
final courseService = CourseService();
try {
  final courseId = await courseService.add(newCourse);
  print('Course created with ID: $courseId');
} on AppException catch (e) {
  print('Error: ${e.message}');
}
```

Listen to Course Updates

```
final courseService = CourseService();

courseService.getAll().listen(
  (courses) {
    print('Courses updated: ${courses.length} courses');
    for (var course in courses) {
      print('- ${course.name} at ${course.time}');
    }
  },
  onError: (error) {
    print('Error: $error');
  },
);
```

Get Homeworks for a Course

```
final homeworkService = HomeworkService();

homeworkService.getHomeworksByCourse('course123').listen(
  (homeworks) {
    print('Homeworks for this course: ${homeworks.length}');
    for (var hw in homeworks) {
      print('- ${hw.title} (Due: ${hw.deadline})');
    }
  }
);
```

```
    },  
  );  
}
```

Mark Homework as Complete

```
final homeworkService = HomeworkService();  
  
await homeworkService.toggleStatus('homework456', true);  
print('Homework marked as done!');
```

Update a Course

```
final courseService = CourseService();  
  
// 1. Get existing course  
final course = await courseService.getById('course123');  
  
// 2. Create modified copy  
final updatedCourse = course?.copyWith(  
  teacher: 'Dr. Anderson',  
  time: '09:00',  
);  
  
// 3. Save changes  
if (updatedCourse != null) {  
  await courseService.update(updatedCourse);  
  print('Course updated!');  
}
```



Advanced Topics

Real-Time Listeners Best Practices

```
// ✅ Good: Unsubscribe when widget is destroyed  
class CourseListScreen extends StatefulWidget {  
  @override  
  State<CourseListScreen> createState() => _CourseListScreenState();  
}  
  
class _CourseListScreenState extends State<CourseListScreen> {  
  late StreamSubscription<List<Course>> _subscription;  
  
  @override  
  void initState() {  
    super.initState();  
  }  
}
```



```
        _subscription = CourseService().getAll().listen((_) {});
    }

    @override
    void dispose() {
        _subscription.cancel(); // ✅ Important!
        super.dispose();
    }
}

// ✅ Better: Use StreamBuilder (auto-managed)
StreamBuilder<List<Course>>(
    stream: CourseService().getAll(),
    builder: (context, snapshot) {
        // StreamBuilder handles subscription lifecycle
    },
)
```

Batch Operations

```
// Add multiple courses at once
final courses = [
    Course(id: '', name: 'Math', teacher: 'Smith', day: 'Mon', time: '08:00'),
    Course(id: '', name: 'English', teacher: 'Jones', day: 'Tue', time: '09:00'),
];

final courseService = CourseService();
for (var course in courses) {
    await courseService.add(course);
}
```

Filtering and Sorting

```
// Client-side filtering (after receiving stream)
courseService.getAll().map((courses) {
    return courses
        .where((c) => c.teacher.contains('Smith'))
        .toList();
}).listen((filtered) {
    print('Filtered courses: $filtered');
});

// Server-side filtering (more efficient)
courseService.getCoursesByDay('Monday').listen((courses) {
    print('Monday courses: $courses');
});
```

Troubleshooting

Common Issues

Issue: Firebase not initialized

```
FirebaseException: [core/not-initialized] Firebase has not been correctly initialized.
```

Solution: Ensure `google-services.json` is in `android/app/` and `flutterfire configure` was run.

Issue: Package name mismatch

```
Caused by: java.lang.Exception: Failed to load FirebaseOptions from resource
```

Solution: Verify `google-services.json` package name matches `applicationId` in `build.gradle.kts`

Issue: Firestore permission denied

```
[cloud_firestore/permission-denied] The caller does not have permission to execute the specified operation.
```

Solution: Update Firestore rules to allow read/write:

```
allow read, write: if true; // Development only
```

Dependencies

```
dependencies:  
  flutter: sdk: flutter  
  firebase_core: ^2.24.0      # Firebase initialization  
  cloud_firestore: ^6.3.0     # Firestore database
```

License

This project is part of the academic management system course project.

Contributing

Contributions are welcome! Please follow these guidelines:

1. Create a feature branch
 2. Make changes with clear commit messages
 3. Test on both Android and iOS
 4. Submit pull request
-

Support

For issues or questions:

- Check [Troubleshooting](#) section
 - Review Firebase documentation: <https://firebase.google.com/docs/flutter/setup>
 - Open an issue on GitHub
-

Last Updated: May 3, 2026

Version: 0.1.0

Maintainer: Development Team